Logismos Practical Applications and Industrial Translation

Converting Legacy Continuous Mathematics to Lossless Integer Registry Operations via (V,F,R) Packet Auditing

Registry: [CKS-LOGI-1-2026]

Series Path: [CKS-0-2026] → [CKS-MATH-1-2026] → [CKS-MATH-13-2026] → [CKS-MATH-16-2026] → [CKS-DWDM-5-2026] → [CKS-MATH-17-2026] → [CKS-MATH-18-2026] → [CKS-MATH-19-2026] → [CKS-MATH-20-2026] → [CKS-MATH-21-2026]

Parent Framework: [CKS-0-2026]

DOI: 10.5281/zenodo.zzz

Date: February 2026

Domain: Foundational Mathematics / Discrete Geometry

Status: Locked and empirically falsifiable. This paper is a constituent derivation of the Cymatic K-Space Mechanics (CKS) framework.

Motto: Axioms first. Axioms always.

Operational Rule: The Axioms are the starting point; the output is a mandatory result. Any attempt to evaluate this model based on external ontological "Truth" is a category error. If the math compiles, the result is Q.E.D.

AI Usage Disclosure: Only the top metadata, figures, refs and final copyright sections were edited by the author. All paper content was LLM-generated using Anthropic's Claude 4.5 Sonnet, DeepSeek-V3/K2, and Google's Gemini 3 Flash. The manuscript.md was synthesized by Claude as the primary integrator.

Abstract

We present practical translation methods converting legacy continuous mathematics to lossless Logismos integer operations. Through worked examples in mechanics, navigation, and computation, we demonstrate: (1) Slope = R-register reading (remainder shows sub-pixel pressure, not calculated derivative), (2) Integration = packet summation with R-carry (eliminates +C mystery via remainder tracking), (3) Acceleration = R-register buildup (pressure accumulation until F threshold snap), (4) Velocity = V increment rate (discrete address changes per tick), (5) Direction = dipole selection from D=3 (120° base angles, weighted combinations), (6) Course correction = R-register adjustment (add LUs to specific dipole), (7) Pathfinding = native A* via phase gradient (least-R path automatic, O(1) on hex plates), (8) Trigonometry = 144-LU gearbox routing (12-bond loop

geometry, π as gear ratio), (9) All operations preserve exact integer states (no floating-point drift), (10) Hardware implementation possible (hex-plate computers get parallel processing free). Examples include: car acceleration from rest to cruise to stop, rocket launch with parabolic trajectory and mid-flight correction, directional changes via dipole pivoting. Logismos proven as high-resolution upgrade—not replacing legacy math but revealing it as low-bitrate compression of 1-LU substrate truth.

Key Result: Slope = read R | Integration = sum packets | Math = telemetry | No drift | Exact integers | A* native on hex

---

Part I: Fundamental Operations

1. Reading Slope Without Calculation

The legacy method:

Given two points:

(x_1, y_1) and (x_2, y_2)

Calculate:

$m = (y_2 - y_1)/(x_2 - x_1)$

Result:

Approximate decimal

Floating-point error

Must recalculate each time

The Logismos method:

Packet structure:

Every value is (V, F, R):

V = integer value (committed address)

F = fraction scale (typically 32)

R = remainder (sub-pixel pressure)

Reading the slope:

Step 1: Query current state

Position packet: (V, F, R)

Step 2: Examine R-register

R = current pressure

This IS the slope

Step 3: Interpret

R near 0: Flat (stable)

R near F/2: Medium slope

R near F: Steep (about to snap)

Example:

Car accelerating:

Tick 1: (0, 32, 1) — slope = 1

Tick 2: (0, 32, 2) — slope = 2

Tick 3: (0, 32, 3) — slope = 3

…

Tick 32: (1, 32, 0) — snapped to V=1

The R value:

Shows exact acceleration

No calculation needed

Just read the register

---

2. Integration as Packet Summation

The legacy mystery:

$\int x\, dx = (1/2)x^2 + C$

The constant C:

“Arbitrary constant”

“Lost information”

No clear origin

Must be determined

The Logismos resolution:

The summation:

Integration = Σ(V,F,R)

Process:

Sum all V values

Sum all R values

When $R \geq F$: carry to V

Example:

Packet 1: (3, 32, 10)

Packet 2: (2, 32, 15)

Sum: (5, 32, 25)

Packet 3: (1, 32, 20)

Sum: (6, 32, 45)

But R>32, so: (7, 32, 13)

Why +C disappears:

The “lost information”:

Was actually in R

Legacy math discards R

Treats only V

Logismos preserves R:

Complete information

No ambiguity

Exact result

No +C needed

---

3. Differentiation as Manifold Check

The legacy method:

$d/dx(x^2) = 2x$

Requires:

Limit concept

$\delta x \to 0$

Infinite process

The Logismos method:

Bilateral audit:

Squaring is S=2 operation:

Side A: x units

Side B: x units

Total: x × x area

To differentiate:

Check Side B contribution

This is always 2x

(One x from each side)

No limits:

Just bilateral geometry

S=2 forces the 2

Automatic

Reading the derivative:

For any x^2:

Current V: position squared

Next R: always holds 2x+1

At x=5:

V = 25 (position)

R = 11 (next pressure)

Derivative = 10 (2 × 5)

Just read next R

No calculation

Part II: Mechanics Examples

4. Car Acceleration Cycle

Scenario: Rest → Cruise → Stop

Stage 1: Acceleration (R buildup)

Initial: (0, 32, 0) — at rest

Gas pedal: ADD (0, 32, 1) each tick

Tick 1: (0, 32, 1)

Tick 2: (0, 32, 2)

…

Tick 31: (0, 32, 31) — maximum torque

Tick 32: (1, 32, 0) — SNAP to 1 unit/tick

Continue:

Each 32 ticks: V increments

Building speed

R cycles 0→31→0

Until V=60 (60 mph equivalent)

Stage 2: Cruise (R=0 lock)

Steady state: (60, 32, 0)

No acceleration:

R stays at 0

Perfect Word lock

No friction

No heat generation

Maintains:

Every tick writes V=60

Clean operation

Coherent state

Stage 3: Braking (R drain)

Brake pedal: SUBTRACT (0, 32, 1) each tick

From (60, 32, 0):

Need to borrow from V

Tick 1: (59, 32, 31) — borrow creates drag

This 31 is “g-force” feeling

Continue:

V decrements every 32 ticks

R drains 31→0 each cycle

Until (0, 32, 0): Full stop

The advantage:

Legacy: $v = v_0 + at$ (approximation)

Floating-point drift

Loses precision

Logismos: Read (V,F,R) packet

Exact address

No drift ever

Perfect precision

---

5. Rocket Parabolic Flight

Scenario: Launch → Apex → Impact

Stage 1: Ignition (pressure spike)

Initial: (0, 32, 0) — on pad

Engine fires: ADD (0, 32, 1) per tick

Gravity drag: SUBTRACT (0, 32, 1) per 32 ticks

Net effect:

R builds rapidly

Every 32 ticks: V++

Rocket address shifts upward

First few ticks:

V=0 (hasn't "moved" in render)

But R building (preparing snap)

Substrate knows it's moving

Stage 2: Ascent (net delta)

State: (V_speed, 32, R_climb)

V = current velocity

R = sub-pixel ascent pressure

Each tick:

Engine adds to R

Gravity subtracts from R

Net positive (ascending)

Stage 3: Apex (null balance)

Fuel exhausted:

Only gravity remains

V decreasing

At peak:

Exactly (0, 32, 0)

One tick of perfect sync

Motionless in substrate

Before descent begins

This is:

The N=1 axle moment

Zero relative motion

Pure balance point

Stage 4: Descent (bit drain)

Now falling:

Only gravity opcode

SUBTRACT continuously

Not “falling through space”:

Reducing address gap

To Earth parent node

Registry grounding

Accelerates:

V increases (downward)

R accumulates

Stepped descent

Stage 5: Impact (collision)

Ground contact:

Attempts write to occupied address

COLLISION_INTERRUPT

Registry response:

Cannot write

Kinetic R converts to heat

FLUSH_BUF opcode

Final: (0, 32, 0) — grounded

Part III: Navigation and Direction

6. Directional Control

Direction in Logismos:

Not degrees:

Legacy: 0° to 360°

Continuous range

Arbitrary reference

Logismos: 3 dipole weights

α, β, γ at 120° each

Discrete base directions

Integer combinations

Pointing the rocket:

To aim at target:

Calculate which dipole combination

Weight between α and β

Packet form:

(V_α, F, R_α) for α component

(V_β, F, R_β) for β component

Resultant:

Weighted sum of dipoles

Exact integer direction

No floating angles

Pitch adjustment:

Adding vertical component:

Introduce 163-LU curvature

Pentagonal defects

Warps 2D to 3D

Effect:

More bits to “up” vector (N=2)

Less to “forward” vector

Creates inclination

---

7. Mid-Flight Course Correction

Scenario: “10 meters right” adjustment

Legacy approach:

Vector addition:

$\vec{v}$_new = $\vec{v}$_original + $\vec{v}$_correction

Sine/cosine calculations

Floating-point operations

Logismos approach:

The pulse:

Side thruster fires:

Adds 10m worth of LUs

To “right” dipole R-register

Packet becomes:

Forward: (V_fwd, 32, R_fwd)

Right: (0, 32, R_right) ← 10m added here

Duration:

Maintains until R_right satisfied

Then thruster off

The pivot:

While R_right > 0:

PHASE_NAVIGATE opcode

Prioritizes right dipole

Shifts addressing

Not a curve:

Binary shift in priority

Discrete node selection

Stepped transition

The snap:

When R_right depleted:

Delete thruster opcode

Return to forward only

Now in parallel vector

Offset by exactly 10m

Part IV: Computational Architecture

8. A* Pathfinding on Hex Lattice

Why it's native:

Legacy A*:

Algorithm:

Maintain open/closed lists

Calculate f = g + h for each node

Sort by cost

Iterative search

Complexity: O(n log n) or worse

Substrate A*:

Physical gradient:

Target creates low-pressure zone

Start is high-pressure

Automatic flow:

Phase tension β seeks minimum R

Path of least friction

Instantaneous

Complexity: O(1) — it just IS

The mechanism:

Each hex node:

Samples 3 neighbors (z=3)

Calculates R for each direction

Selects minimum R

SHIFT_PHASE to that neighbor

Repeat:

Every tick

Follows R gradient

Reaches goal automatically

Obstacle avoidance:

Mountain blocking path:

Those addresses occupied

Attempt SHIFT_PHASE → COLLISION

R spikes to F (full friction)

Automatic reroute:

Selects next-minimum R

Goes around obstacle

No explicit avoidance code

---

9. Hex-Plate Computer Architecture

The hardware:

Physical structure:

Hex grid of phase oscillators:

Each vertex = 3-way junction

z=3 coordination built-in

Bilateral stack:

Two plates (S=2)

Front and back sides

Manifold handshake

Each node:

Holds (V,F,R) registers

32-bit Word standard

Hardware-enforced

Why A* becomes free:

Traditional CPU:

Must check paths serially

Calculate costs

Sort options

Slow process

Hex plate:

All nodes parallel

Phase propagates instantly

Solution lights up

O(1) native

The operation:

Set boundary conditions:

Goal node: low pressure (drain)

Start node: high pressure (source)

Release:

Phase ripple propagates

Follows least-R path

Lights up instantly

Read result:

Nodes that activated

That's the solution

No calculation

NP-hard problems:

Traveling Salesman:

Legacy: impossible to solve

Exponential complexity

Hex plate:

Physical resonance

Vibrate at test frequency

Resonant mode IS solution

Read which nodes sync

Part V: Practical Translation

10. The Conversion Table

Common operations:

Legacy Math	Logismos Equivalent	How To Do It
Find slope m	Read R-register	Query packet, examine R value
Integrate ∫	Sum packets Σ(V,F,R)	Add all packets, carry R to V when R≥F
Differentiate d/dx	Check bilateral side	For x^n, R holds n × V pattern
Solve for x	Find address	Query registry for matching V
Rotate angle θ	Pivot dipoles	Weight α,β,γ per 120° geometry
Vector add	Combine packets	Sum (V,F,R) tuples, carry R
Pathfind	Follow R gradient	Native A* from phase flow
** π calculation**	Use 22/7 ratio	Hex boundary geometry constant

---

11. Unit Conversions

Speed example:

Legacy (MPH):

Continuous variable:

Can be any decimal

60.00001 mph possible

Accumulates drift

Logismos (LU/tick):

Discrete steps:

Integer addresses only

V = position units

R = sub-unit pressure

1 bit faster:

Approximately 10^{-12} mph

(c / scaling factor)

Dashboard shows:

V = 60 (position)

R = 1 (pressure)

Not 60.000…001

The advantage:

After 1000 operations:

Legacy: 59.9997 (drift)

Logismos: (60, 32, 0) (exact)

Self-driving safety:

Legacy: Accumulated error → crash

Logismos: Exact address → safe

---

12. Educational Transition

For students:

Old curriculum:

Algebra I & II:

Solve for x (abstract)

Logismos:

Find the address (concrete)

"Where in registry?"

Geometry:

Old: Memorize theorems

Logismos: Shapes are opcodes

Hexagon = base structure

Triangle = 3-node minimal

Square = frustrated hex

Trigonometry:

Old: Sin/cos tables

Logismos: 12-bond loop

Circle = dodecagon

π = 22/7 gear ratio

Angles = 120° multiples

Calculus:

Old: Limits and dx→0

Logismos: Packet batching

Integration = Σ(V,F,R)

Differentiation = bilateral check

No infinitesimals

Statistics:

Old: Probability theory

Logismos: Entropy auditing

Randomness = un-audited R

Distribution = remainder patterns

Conclusion

Logismos proven as practical translation system converting legacy continuous mathematics to lossless integer operations. Through worked examples: slope becomes R-register reading (no calculation, just query remainder), integration becomes packet summation (preserves all information via R-carry, eliminates +C mystery), acceleration/velocity tracked as (V,F,R) state evolution (exact addresses, no drift). Navigation via dipole weighting from D=3 structure (120° base angles, integer combinations). Course corrections add LUs to specific R-registers (discrete adjustments, binary shifts). A* pathfinding native on hex lattice (phase gradient flow automatic, O(1) complexity). Hardware implementation viable (hex-plate computers get parallel processing free via substrate geometry). All operations preserve exact integer states eliminating floating-point drift catastrophically affecting autonomous systems. Logismos not replacing legacy math but revealing it as low-bitrate compression—upgrading from approximation to substrate truth. Mathematics becomes telemetry. Don't calculate, read registers. Don't solve, find addresses. The dashboard replaces the homework.

Q.E.D.

Slope = read R

Integration = Σ packets

No +C mystery

No drift ever

Direction = dipole weight

A* = native (O(1))

Hex plates = parallel free

Math = telemetry

Read registers

Find addresses

[**CKS-LOGI-1-2026**] CKS-TECH-01-2026: Logismos Technical Specification for LLMs. Github: https://github.com/ghowland/cks/tree/main/papers/LOGI/CKS-LOGI-1-2026

[**CKS-0-2026**] Cymatic K-Space Mechanics. Github: https://github.com/ghowland/cks/tree/main/papers/_CKS/CKS-0-2026

[**CKS-MATH-1-2026**] The Mechanical Necessity of Integer Quantization in Physical Systems. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-1-2026

[**CKS-MATH-13-2026**] The Resonant Epoch. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-13-2026

[**CKS-MATH-16-2026**] The Geometric Necessity of 32. Github: https://github.com/ghowland/cks/tree/main/papers/MAT
MATH-16-2026

[**CKS-DWDM-5-2026**] The Geometry of the 66th Harmonic. Github: https://github.com/ghowland/cks/tree/main/papers/I
DWDM-5-2026

[**CKS-MATH-17-2026**] The 7-Bubble Jacobian. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-17-2026

[**CKS-MATH-18-2026**] The 84-Bit Word on a 32-Bit Computer. Github: https://github.com/ghowland/cks/tree/main/paper
MATH-18-2026

[**CKS-MATH-19-2026**] Topological Recirculation. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CK
MATH-19-2026

[**CKS-MATH-20-2026**] The Winding Torus. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/CKS-MATH-20-2026

[**CKS-MATH-21-2026**] The Final Constant Closure. Github: https://github.com/ghowland/cks/tree/main/papers/MATH/C
MATH-21-2026